

MILESTONE 1

In Milestone 1, we decided to make an application that focused on donating and claiming food in smaller communities instead of larger cities. We initially decided to make this service a centralised platform that can be accessed on all devices to provide ease of use for the community.

We were largely successful in implementing the ability to create food posts, allow reservations, and create and join groups. Our frontend is also accessible and easy to use, creating a UI that makes the application inclusive for all users. As planned, we also included tags that allow users to filter food to exclude allergens and view posts by their expiry dates.

We were successful in incorporating features that solve the problem from Milestone 1. However, the lack of an automatic expiry system limits reliability. This would be prioritised in a second prototype by adding a scheduling algorithm that updates the statuses of food items daily removing those that have expired at midnight.



OPENPANTRY

MILESTONE 3

The features we focused on in our first prototype were the ones most beneficial to have in a Minimum viable product. This includes user accounts, creating and reserving food posts, communities, and a simple and accessible user interface. Minor adjustments were made without affecting core functionality. We had decided against including the announcements feature since it is not needed for a first prototype.

We had used most of the technologies that we had planned to use in Milestone 3: React and SpringBoot. These technologies were appropriate given existing team familiarity, along with their usefulness within full-stack development. This allowed development time to be prioritised over learning new technologies. However, we had to switch from PostgreSQL to H2 due to its simplicity and faster setup.

If we were to do things differently, we might use Next.js to create a multi-page front-end for our system, and to also improve the user experience and the overall look of the system, especially by including correct validation errors.

MILESTONE 2

We have implemented user accounts, management of creating and joining groups, posting and claiming food items, a tagging and filtering system, as well as a simple UI and accessibility functions.

Convenience features such as a QR code generator, moderation tools, announcements, notifications, and a chat feature were omitted. These were excluded as they were non-essential to the core functionality and beyond the scope of the first prototype, limiting overall usability.

A key change was allowing users to join communities instantly, rather than requiring approval. This was for ease of access when testing the prototype.

For a second prototype, we need to improve moderation in communities by implementing features previously discarded; invite systems, approval-based access, and moderation tools such as banning and kicking users. We could also add a chat feature to improve user coordination within the application.

WHAT IS OPENPANTRY?

OpenPantry is a web application that allows people to come together to donate and claim food within their local communities. Users are able to join communities that are close to them. They can upload food posts into these communities with an image and description of said food, along with an expiration date and tags that can be filtered out. The application has many accessibility features to support a wide range of users, including those with varying technical abilities or specific needs.

It focuses on target 11.b of the UN Sustainable Development Goal, as it increases resource efficiency by ensuring food that would be wasted is instead consumed by others. By redistributing surplus food that would otherwise be discarded, it aligns with target 11.6, reducing the environmental impact associated with disposal.

MILESTONE 4

In the actual development stage of making our software, development was structured around individual ownership of system entities from the ER diagram. Initially, we were all meant to be working on the back-end, but due to features being cut and therefore fewer entities, we split up into a back-end and front-end team.

In the back-end, we handled a single entity from the diagram, where we would individually complete the different layers (service, controller, etc.), and we would write our tests separately. Work was conducted on separate GitLab branches. Once we had finished, we assessed and merged all the branches into the master branch.

The front-end was developed similarly, where group members developed different parts of the UI. Individual components, such as the login page, communities, accessibility, etc., were developed and tested separately before being merged into the main application.

Integration presented challenges, particularly due to naming inconsistencies and merge conflicts. However, once resolved, all our individual efforts into creating the features allowed us to have a working back-end. When combined with the front-end, this resulted in a fully functional application.